

Software Requirements Specification

FindIT: Smart Lost and Found Management System

Team NexuX

Date: February 16, 2026

Organization

Department of Computer Engineering
Faculty of Engineering, University of Peradeniya

Course

CO2060 – Software Systems Design Project

Team Members

Dulmina Weerasinghe
Livindu Jayasinghe
Lihini Silva
Thenuk Piyathilake

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Scope	2
1.3	Product Overview	2
1.3.1	System Context	2
1.3.2	Product Functions	2
1.3.3	User Characteristics	3
1.3.4	Limitations	3
1.4	Definitions, Acronyms, and Abbreviations	3
1.5	References	3
2	Specific Requirements	4
2.1	External Interface Requirements	4
2.1.1	User Interfaces	4
2.1.2	Hardware Interfaces	4
2.1.3	Software Interfaces	4
2.2	Functional Requirements	4
2.2.1	Feature A: Secure Authentication	4
2.2.2	Feature B: Reporting Items	4
2.2.3	Feature C: Intelligent Matching	4
2.2.4	Feature D: Mutual Verification	5
2.2.5	Feature E: Lifecycle Management	5
2.3	Non-Functional Requirements	5
2.4	Usability and Human-System Interface	5
2.5	Security Requirements	5
2.6	Safety Requirements	5
2.7	Design and Implementation Constraints	5
3	Verification	6
3.1	Inspection	6
3.2	Analysis	6
3.3	Demonstration	6
3.4	Test	6
4	Appendices	7
4.1	Assumptions	7
4.2	Dependencies	7

1 Introduction

1.1 Purpose

The purpose of this document is to specify the functional and non-functional requirements for the A University Lost and Found Management System - FindIT. This system aims to centralize and streamline the process of reporting, matching, and recovering lost personal items within a shared environment. It is intended for project stakeholders, developers, and system administrators to ensure a shared understanding of the system's goals and constraints.

1.2 Scope

FindIT is a digital platform designed to replace fragmented, informal recovery methods (like physical notice boards and social media chats) with a structured, privacy-first matching engine.

- **What the software will do:** Provide structured reporting for lost and found items, utilize an intelligent decision-support engine to suggest potential matches, and facilitate secure, gradual communication between parties.
- **Business Value:** The system reduces the "cognitive burden" on users, increases the success rate of item recovery, and enhances the Faculty security by providing a centralized, data-driven overview of loss patterns while protecting user privacy.

1.3 Product Overview

1.3.1 System Context

FindIT operates as a standalone web application designed for institutional use (e.g., a faculty, university). It acts as a digital intermediary between the person who lost an item and the person who found it. While it integrates into the university ecosystem, it does not manage physical inventory or legal custody of items; it functions strictly as an information exchange and matching layer.

1.3.2 Product Functions

- **Structured Reporting:** Questionnaire-based submission for lost and found items.
- **Decision-Support Matching:** Automated comparison of item attributes to flag potential matches for human review using similarity algorithms.
- **Visibility Control:** Restricted public views of found items to prevent fraudulent claims.
- **Gradual Disclosure Messaging:** A controlled communication channel that allows users to verify ownership before sharing personal contact details.
- **Notification System:** Real-time alerts for potential matches and status updates.

1.3.3 User Characteristics

- **The Loser:** Students, faculty, or visitors who have lost an item. They may be under emotional stress and require a simple, reassuring interface.
- **The Finder:** Individuals who wish to do the right thing with minimal friction. They require a quick way to report items without public exposure.

Note: All users are assumed to have basic digital literacy but no specialized technical expertise.

1.3.4 Limitations

- **No Physical Custody:** The system does not track where an item is physically stored.
- **No Legal Authority:** The software cannot legally prove ownership or resolve disputes.
- **Description Ambiguity:** Matching accuracy is limited by the quality of user-provided descriptions.
- **Privacy-Matching Tradeoff:** Highly restrictive visibility may slightly slow down the matching process.

1.4 Definitions, Acronyms, and Abbreviations

- **LFMS:** Lost and Found Management System.
- **Gradual Disclosure:** Revealing item details in stages to verify claimant legitimacy.
- **OTP:** One-Time Password.
- **Python Fuzzy Logic:** A library (e.g., FuzzyWuzzy) used for string similarity matching and semantic comparison.
- **PWA:** Progressive Web App.
- **9:16:** Vertical aspect ratio prioritized for mobile screens.

1.5 References

- Project Context Document: Requirement Analysis (2026).
- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications.

2 Specific Requirements

2.1 External Interface Requirements

2.1.1 User Interfaces

- **Visual Style:** UI shall feature a modern, distinct design language (FindIT branding).
- **Public Dashboard:** Restricted "View-Only" mode accessible without login.
- **Verification Room:** Private interface for matched users and security questions.

2.1.2 Hardware Interfaces

- **Mobile Camera:** System shall interface with device camera for capturing item images.

2.1.3 Software Interfaces

- **Authentication Gateway:** SMTP Relay to send OTPs to university emails.
- **Matching Engine:** Python Fuzzy Logic library for string similarity analysis and attribute matching.
- **Web Push Notifications:** Service Worker API for real-time alerts.

2.2 Functional Requirements

2.2.1 Feature A: Secure Authentication

- **REQ-FN-001:** Only allow emails ending in @eng.pdn.ac.lk.
- **REQ-FN-003:** Verify identity via 6-digit OTP.

2.2.2 Feature B: Reporting Items

- **REQ-FN-005:** Finders must define a secret challenge based on a visible detail.
- **REQ-FN-006:** Owners must define a secret challenge based on a memory detail.
- **REQ-FN-008:** Hide contact numbers until the system verifies challenge answers from both parties.

2.2.3 Feature C: Intelligent Matching

- **REQ-FN-009:** Use Python Fuzzy Logic for matching based on Category, Brand, Color, and Location.
- **REQ-FN-010:** Group multiple potential matches (Similarity Score > 80%) into a Match List.

2.2.4 Feature D: Mutual Verification

- **REQ-FN-014:** The Fuzzy Logic engine shall evaluate the similarity between user-submitted answers and the stored challenge keys.
- **REQ-FN-017:** Lock match session after 3 incorrect attempts.

2.2.5 Feature E: Lifecycle Management

- **REQ-FN-021:** Auto-discard report if no response within 48 hours of a 7-day alert.
- **REQ-FN-022:** Implement 72-hour Match Expiry timer for unverified matches.

2.3 Non-Functional Requirements

- **Security:** Contact numbers shall be stored in an encrypted format.
- **Performance:** Local matching routine shall complete in under 1 second.
- **Scalability:** Support concurrent usage by at least 500 active students.
- **Reliability:** 99% uptime during the academic semester.

2.4 Usability and Human-System Interface

- **2.4.1 Accessibility:** Design for high contrast and readability.
- **2.4.3 Visual Cues:** Use Red for Lost and Green for Found.

2.5 Security Requirements

- **2.5.2 Authorization:** Restrict contact info until Secret Challenge is solved.
- **Data Integrity:** All sensitive data stored using relational data encryption.

2.6 Safety Requirements

- **2.6.1 Fraud Mitigation:** Hide exact location details from public dashboard.
- **2.6.2 Session Expiry:** Auto-clear cache after 20 minutes of inactivity.

2.7 Design and Implementation Constraints

- **Mobile-First:** UI design must prioritize 9:16 aspect ratio.
- **Local Processing:** String analysis and similarity matching must be performed on the local server to reduce external API dependency.

3 Verification

3.1 Inspection

- **REQ-2.1.1:** Inspection of CSS to ensure FindIT branding is applied.
- **REQ-2.8.3:** Inspection of media queries for 9:16 aspect ratio.

3.2 Analysis

- **REQ-2.3.2:** Analysis of DB indexing to prove support for 500 users.
- **REQ-2.8.1:** Analysis of local server CPU/Memory usage during Fuzzy matching routines.

3.3 Demonstration

- **REQ-2.1.2:** Demonstration of Found Item form triggering device camera.

3.4 Test

- **REQ-2.2.1:** Test registration with non-faculty email (should fail).
- **REQ-2.2.3:** Input "Black Wallet" vs "Dark leather purse" to test similarity score accuracy of the Fuzzy Logic engine.
- **REQ-2.3.1:** Use performance tools to verify matching under 1 second.

4 Appendices

4.1 Assumptions

- Users act in good faith and provide truthful descriptions.
- Users have access to @eng.pdn.ac.lk email accounts.

4.2 Dependencies

- Stability of University Wi-Fi (PE-Net).
- Availability of the Python FuzzyWuzzy/Levenshtein library within the server environment.
- Browser support for PWA Standards.

— END OF DOCUMENT —